# INTELLIDCE: END-TO-END SYSTEM ARCHITECTURE SPECIFICATION

*Comparative Architectural Blueprint: Classical SSA Baseline vs. Proposed 7-Module Agentic & Secure Pipeline*

## EXISTING BASELINE ARCHITECTURE

(Classical Heuristic SSA Optimization Pipeline)

### 1. C/C++ Source Input

*Raw User Program*

- Single compilation unit (.cpp / .c)
- No pre-execution security verification

### 2. GCC Front-End & GIMPLE SSA

*GCC -O0 -fdump-tree-ssa Ingestion*

- Translates AST to 3-address GIMPLE code
- Exposes explicit phi-nodes (Φ) and blocks
- Dumps raw static text files (.ssa)

### 3. Lexer & Parser Layer

*src/lexer.l (Flex) & src/parser.y (Bison)*

- Flex scans tokens (opcodes, variables)
- Bison LALR grammar validates syntax
- Builds in-memory ProgramIR & FunctionIR

### 4. Classical Heuristic DCE Engine

*src/analysis.cpp (Heuristic Passes)*

- Sparse Conditional Constant Propagation
- BFS/DFS unreachable block pruning
- Local dead store elimination (Def-Use)
- Basic phi-node & control edge cleanup

### 5. Static File Emission

*Direct Disk Output*

- Emits optimized.ssa text file
- Generates graph.dot for Graphviz
- Produces summary.txt with basic counts

### 6. Direct Host Web UI

*Express + Static HTML/D3.js*

- Direct local host execution (unsandboxed)
- D3.js interactive CFG visualization
- Shows raw before/after code panes

### CRITICAL SYSTEM LIMITATIONS

*Architectural Deficiencies Identified:*

- Strict heuristic limits: misses semantic dead code
- Zero security awareness: sensitive dead stores ignored
- Black-box elimination: no developer explanation
- Unsandboxed execution: host vulnerable to exploits
- No verification: semantic breakage undetected
- No feedback loop: static model never improves

## PROPOSED ENHANCED SYSTEM ARCHITECTURE (FULL 7-MODULE VISION)

(End-to-End Secure, Agentic AI, Neural Optimization & Verification Framework)

### Source Code (C/C++)

*User Input Buffer*

- Pasted source or uploaded files
- Direct multi-module entrypoint

Pre-Scan

### Vulnerability Pattern Scanner

Module 2

*Regex & AST Pattern Matching Engine*

- CWE-561: Dead memory stores to credentials
- CWE-570/571: Invariant always-false control gates
- CWE-670: Dormant hazardous APIs (system, strcpy)
- CWE-252: Unchecked return value vulnerabilities

### SSA Extraction & Lexer/Parser

*GCC GIMPLE + Flex/Bison Pipeline*

- GCC generates explicit SSA 3-address code
- Flex tokenizer & Bison LALR grammar
- Constructs enriched CFG & Def-Use call chains

### 74-D Neural Graph Representation

*Hybrid Structural & Semantic Embedding*

- 10 Structural Features: loop depth, post-dom, def-use span
- 64-D Semantic Embeddings via MiniLM-L6-v2 transformer
- Constructs PyTorch Geometric HeteroData Graph tensor
- Captures both syntactic topology and variable semantics

Feedback Retrain Loop

### Hybrid SAGEConv + GATConv GNN Core

*Deep Neural Optimization Predictor*

- Inductive neighbor aggregation via GraphSAGE layers
- Multi-Head Attention mechanism (GATConv) across CFG
- Learns complex inter-block deadness dependencies
- Outputs per-node elimination probability P(dead)

P(dead)

### ML Confidence Filter & Active Learning

Module 7

*Uncertainty Quantification & Retraining Loop*

- Confidence Gate: P(dead) > 0.85 (High), P < 0.15 (Live)
- Ambiguity Zone (0.45 <= P <= 0.65): Triggers HITL Queue
- Human-in-the-Loop review for anomalous edge cases
- Retraining Pipeline: logs corrections to active dataset
- Iterative model fine-tuning prevents systematic drift

Approved Candidates

### Safety Veto & Bug Triage Engine

Module 3

*Asymmetric Safety Verification Layer*

- Strict Side-Effect Auditor: volatile memory, I/O, syscalls
- Recursion Trap Resolver & infinite loop culling fix
- Def-Use invariant checking & memory safety guardian
- Rule of Safety: Heuristic veto unconditionally overrides
- Semantic equivalence preservation guarantee

Optimized IR

### Sandboxed Execution Engine

Module 5

*Isolated Micro-Container & Verification Jail*

- Isolated Docker / Seccomp Linux namespace container
- Safe isolated compilation of user C/C++ code
- Executes test suite with strict CPU/memory timeouts
- Captures runtime exit codes, stdout, stderr, memory traps
- Prevents untrusted payloads from compromising host system

Execution Telemetry

Auto-Fix Re-test

### Autonomous Agentic Debugger

Module 6

*Closed-Loop Observe-Diagnose-Suggest-Verify Cycle*

- OBSERVE: Captures test harness failures & regression errors
- DIAGNOSE: Correlates failing test back to eliminated CFG nodes
- SUGGEST: Generates AST/SSA patch & restores required nodes
- VERIFY: Automatically dispatches re-test inside Sandbox
- Self-healing regression loop ensures zero semantic deviation

Verified Metrics

Verified Code & Diffs

### Program Vitality & Health Scorecard

Module 1

*Comprehensive Quality & Security Analytics Dashboard*

- Formula-Driven Vitality Index (0–100 Rating)
- CFG Tangle Factor (Branch-to-Block Cyclomatic Ratio)
- Dead Code Elimination Density (Bytes & Statements)
- Quality Badges: EXCELLENT / GOOD / ATTENTION / CRITICAL
- Security Badges: SECURE / AT RISK / VULNERABLE
- Correlates static CWE triage flags with runtime vitality
- Gives developers instantaneous diagnostic confidence

### Agentic AI DCE Explainer & Dialogue

Module 4

*Multi-Turn Interactive Compiler Assistant*

- Context-Aware Natural Language Explanations
- Root-Cause Semantics: explains WHY code became dead
- Actionable Upstream C++ Refactoring Fix Recommendations
- Interactive Multi-Turn Dialogue for developer queries
- Mode-Aware UX: toggles Classical vs Neural vs Agentic
- Human-in-the-Loop approval console for Module 7 edge cases
- Eliminates compiler opacity through grounded reasoning